

A3

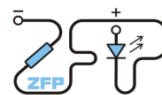
October 19, 2025

```
[1]: # TODO: Replace the string "Your Name" and "DD.MM.YYYY" with the appropriate ↵  
      ↵ values.
```

```
from header import header  
_ = header(student="Jonáš Venc", date="16.10.2025", task_no=3)
```

Kabinet výuky obecné fyziky, UK MFF

Fyzikální praktikum 4



Úloha č. 3

Název úlohy: Identifikace prvků na základě jejich charakteristického rentgenového záření.

Jméno: Jonáš Venc

Datum měření: 16.10.2025

Připomínky opravujícího:

Udělený počet bodů

Výsledky a zpracování měření	
Diskuse výsledků	
Závěr	
Seznam použité literatury	
Celkem	max. 12

Posuzoval:

dne:

1 Identifikace prvků na základě jejich charakteristického rentgenového záření.

2 Pracovní úkoly

1. Proveďte energetickou kalibraci γ -spektrometru pomocí α -zářiče ^{241}Am .
2. Určete materiál několika vzorků.
3. Stanovte závislost četnosti rentgenového záření na atomovém čísle elementu.
4. Určete relativní zastoupení prvků v jednom ze vzorků.

3 Úvod

Deexcitace vzbuzených stavů elektronového obalu je provázena emisí rentgenového záření podle [3]. Protože energie tohoto záření je charakteristická pro daný prvek [1], lze jej využít ke kvalitativní i kvantitativní analýze složení materiálů. Elektronový obal lze excitovat několika způsoby. Rentgenová fluorescenční metoda využívá interakci fotonů s elektrony obalu: při fotoelektrickém jevu jsou elektrony vyraženy a obal se dostává do excitovaného stavu. Následná deexcitace vede k emisi charakteristických rentgenových čar.

V našem experimentu měříme spektra vznikající při ozáření různých vzorků γ -zářením z radioaktivního zdroje. Na základě energií rentgenových přechodů je možné identifikovat přítomné prvky a určit složení vzorku. Ke kvantitativnímu určení je nutné vyhodnotit i intenzity čar, které závisejí nejen na obsahu prvků, ale také na geometrii experimentu, tloušťce a složení vzorku, intenzitě buzení a vlastnostech detektoru. Citlivost metody je dána především energetickým rozlišením a účinností spektrometru.

V našem experimentu je použitým zdrojem γ -záření izotop ^{241}Am . Jádra tohoto izotopu se α -rozpadem přeměňují na jádra ^{237}Np , často ve vzbuzeném stavu. Při jejich deexcitaci vzniká několik γ -linek [2], z nichž nejintenzivnější má energii 59,5 keV. V měřeném spektru se navíc objevuje rentgenové záření dceřinného neptunia, především L-série v oblasti 10–20 keV.

Radioaktivní zdroj je umístěn ve stojánku nad vzorkem, vedle detektoru. Směrem od zdroje k detektoru stojánek funguje jako stínění, zatímco mezi vzorkem a detektorem stínění není. Charakteristické rentgenové záření je detekováno polovodičovým spektrometrem, který se skládá z germaniového detektoru s beryliovým okénkem, nábojově citlivého předzesilovače, lineárního zesilovače, vysokonapětového zdroje a USB datového akvizičního modulu. Detektor pracuje při napětí přibližně 3,5 kV.

Energetické rozlišení spektrometru je $\text{FWHM} \approx 0,7 \text{ keV}$ v oblasti 5–60 keV. Kvůli tomuto relativně špatnému rozlišení a tlustému beryliovému okénku není možné registrovat záření s energiemi pod 4 keV. Proto nelze identifikovat prvky s protonovým číslem $Z < 20$, například příměsi hliníku, a výsledná analýza může být nepřesná.

Vzhledem k energiím rentgenového záření a hustotám používaných vzorků je zřejmé, že zkoumáme jejich složení do hloubky několika desítek až málo stovek μm . Z hloubky větší než tři polotloušťky uniknou maximálně procenta vzniklého záření.

TODO: citujme studijní text.

4 Postup měření

Nakalibrujeme spektrometr radioaktivním zářičem ^{241}Am . Dále naměříme spektra rentgenového záření několika různých vzorků, zpracujeme spektra a identifikujeme materiál či materiály vzorků pomocí tabulek energií rentgenového záření. Dobu jednotlivých měření volíme tak, aby nejistota výtěžku byla menší než 3%.

5 Výsledky měření

Úkol: Nakalibrujte spektrometr pomocí radioaktivního zářiče ^{241}Am . Použijte tabelované energie singletních píků. Systematickou nejistotu kalibrace odhadněte pomocí multipletů L -série neptunia. Diskutujte výslednou kalibraci a její nejistoty.

Řešení:

Systematická chyba je spočtena jako rozdíl centroidu dvou fitovaných multipletů a tabulkové hodnoty.

Naměřená: 17.55; tabulková: 17.8

Naměřená: 20.83; tabulková: 20.8

sigma1 = 0.25 keV

sigma2 = 0.0 keV

```
[14]: sigma1 = 0.25
      sigma2 = 0.03

      sigma_syst = (sigma1+sigma2)/2
      print("Průměrná systematická chyba:")
      print(sigma_syst)
```

Průměrná systematická chyba
0.14

```
[16]: import numpy as np

      fwhm = 1.5
      sigma = fwhm / (8 * np.log(2))**0.5

      chyba = sigma / 7750**0.5
      print("Statistická chyba:")
      print(chyba)
      print("Vidíme, že je vůči systematické zanedbatelná.")
```

Statistická chyba:
0.007235735513364038
Vidíme, že je vůči systematické zanedbatelná.

Úkol: Měřené vzorky jsou na první pohled kovy, předpokládejme, že jsou to buď jednoprvkové vzorky, nebo slitiny dvou prvků a to se $26 \leq Z \leq 52$. Určete chemické prvky v jednodruhových

vzorcích. Vytvořte tabulku s číslem vzorku, naměřenou energií píku, čistou plochou píku, její nejistotou, FWHM, a časem měření. Dále do tabulky přidejte:

- identifikovaný typ spektrální čáry (K_a, K_b, K_a+b, L_a, L_b, ...),
- identifikovaný chemický prvek,
- protonové číslo identifikovaného prvku.

Každý řádek v tabulce odpovídá jedné spektrální čáře daného vzorku. Určete také chemické složení víceprvkových vzorků - slitin. Uveďte je také v tabulce (jeden řádek pro každou spektrální čáru každého prvku). Tabulku, resp. měřené hodnoty dále upravíme do podoby vhodné k další analýze.

Řešení: 1. Vepišme naměřená data.

```
[1]: # TODO: Insert experimental data into the list like so:
# data = [
#     [1, 9.42, 46161, 337, 0.75, 1000.1, "K_a+b", "Ga", 31], # Ga; Z = 31
#     ...
# ]
# Elements of inner list are (from screen or .Rpt files):
# ["ID", "CENTROID", "NET", "+/-", "FWHM", "MT", "Line", "Element", "Z"]
# tzn. ["ID č.", "Energie (keV)", "Net Area", "nej. Net Area", "FWHM", "Meas.
↳Time (s)", "Čára", "Prvek", "Z"]
#
# Notes:
# - "ID" is an integer identifying the sample (1, 2, 3, ...).
# - "Line" supports the following values (case sensitive): "K_a", "K_b",
↳"K_a+b", "L_a", "L_b", ...
# - "MT" is the measurement time in seconds (float). Use the live time value
↳from the software.

data = [
    [1, 8.12, 48491, 570, 0.68, 1441, "K_a+b", "Cu", 29],
    [2, 25.24, 48799, 371, 0.76, 481, "K_a", "Sn", 50],
    [2, 28.55, 11440, 214, 0.79, 481, "K_b", "Sn", 50],
    [3, 20.13, 26516, 264, 0.68, 360, "K_a", "Rh", 45],
    [3, 22.72, 6081, 165, 0.69, 360, "K_b", "Rh", 45],
    [4, 10.57, 5960, 129, 0.56, 203, "L_a", "Pb", 82],
    [4, 12.62, 4710, 154, 0.61, 203, "L_b", "Pb", 82],
    [5, 22.07, 53634, 405, 0.73, 1124, "K_a", "Ag", 47],
    [5, 24.94, 17018, 338, 0.76, 1124, "K_b", "Ag", 47],
    [5, 8.35, 8565, 340, 1.14, 1124, "K_a+b", "Cu", 29],
    [6, 15.74, 54621, 451, 0.7, 884, "K_a", "Zr", 40],
    [6, 17.66, 14669, 369, 0.72, 884, "K_b", "Zr", 40],
    [11, 23.09, 31231, 278, 0.71, 304, "K_a", "Cd", 48],
    [11, 26.11, 8902, 184, 0.79, 304, "K_b", "Cd", 48],
    [9, 17.42, 29456, 281, 0.71, 326, "K_a", "Mo", 42],
    [9, 19.63, 5978, 164, 0.64, 326, "K_b", "Mo", 42],
]
```

2. Vytvořme pandas DataFrame z listu dat.

```
[2]: import pandas as pd

# Create a DataFrame from exp. data, set the column names.
df = pd.DataFrame(data, columns=["ID", "CENTROID", "NET", "+/-", "FWHM", "MT",
↪ "Line", "Element", "Z"])
```

3. Určete (statistickou) nejistotu naměřené energie. Přidejme ji do tabulky jako sloupec s názvem "unc_CENTROID". Poté z tabulky odstraníme dále nepotřebné sloupce.

```
[3]: import numpy as np

# TODO calculate the energy uncertainty:
df["unc_CENTROID"] = df["FWHM"]/2.35/np.sqrt(df["NET"])
# each sample is measured for different time, so we convert to 'per second'
↪ units
df["Intensity"] = df["NET"] / df["MT"]
df["unc_Intensity"] = df["+/-"] / df["MT"]

# we can get rid of columns "NET", "+/-", and "FWHM" as we won't need them
↪ anymore
df = df.drop(columns=["NET", "+/-", "FWHM"])
```

4. Vypišme DataFrame. Pro přehlednost vypišme nejistotu energie píku za energii píku.

```
[4]: # Print out the DataFrame.
# The method to_string() is used to print the DataFrame without the index column
# and to print all rows without breaking the table.
# The col_space parameter sets the minimum column width (in characters).
cols = ["ID", "Element", "Z", "CENTROID", "unc_CENTROID", "Line", "Intensity",
↪ "unc_Intensity"]
headers=["Vzorek", "Prvek", "Z", "E (keV)", "nej. E (keV)", "Čára", "I (s-1)",
↪ "nej. I (s-1)"]
print(df.round(3).to_string(index=False, col_space=7, columns=cols,
↪ header=headers))
```

Vzorek	Prvek	Z	E (keV)	nej. E (keV)	Čára	I (s ⁻¹)	nej. I (s ⁻¹)
1	Cu	29	8.12	0.001	K _{a+b}	33.651	0.396
2	Sn	50	25.24	0.001	K _a	101.453	0.771
2	Sn	50	28.55	0.003	K _b	23.784	0.445
3	Rh	45	20.13	0.002	K _a	73.656	0.733
3	Rh	45	22.72	0.004	K _b	16.892	0.458
4	Pb	82	10.57	0.003	L _a	29.360	0.635
4	Pb	82	12.62	0.004	L _b	23.202	0.759
5	Ag	47	22.07	0.001	K _a	47.717	0.360
5	Ag	47	24.94	0.002	K _b	15.141	0.301
5	Cu	29	8.35	0.005	K _{a+b}	7.620	0.302
6	Zr	40	15.74	0.001	K _a	61.788	0.510

6	Zr	40	17.66	0.003	K_b	16.594	0.417
11	Cd	48	23.09	0.002	K_a	102.734	0.914
11	Cd	48	26.11	0.004	K_b	29.283	0.605
9	Mo	42	17.42	0.002	K_a	90.356	0.862
9	Mo	42	19.63	0.004	K_b	18.337	0.503

Úkol: Diskutujte rozdíly mezi systematickými a statistickými nejistotami naměřených energií. Která z nich je dominantní? Znemožňuje některá z nich, či jejich kombinace jednoznačnou identifikaci prvků?

Řešení:

Systematická chyba je dominantní vůči statistické, proto můžeme statistickou chybu zanedbat (je řádově nižší). Systematická je nejistota vzniklá z principu měření a nemůžeme ji tedy snížit zvýšením počtu opakování měření oproti statistické. Obecně jejich kombinace se počítá jako součet jejich druhých mocnin pod odmocninou. Přesto jsme i s těmito chybami určit jednoznačnou identifikaci prvků.

Úkol: Vytvořte tabulku, kde porovnáte měřené energie s tabelovanými.

```
[5]: # Solution

# TODO add more elements/lines to the dictionary `theory` if need be.

# Minimal theory table (keV).
theory = {
    'Cu': {'Ka1': 8.046, 'Ka2': 8.026, 'Kb1': 8.904, 'Kb2': 8.976, 'Kb3': 8.904},
    'Sn': {'Ka1': 25.267, 'Ka2': 25.040, 'Kb1': 28.481, 'Kb2': 29.104, 'Kb3': 28.439},
    'Rh': {'Ka1': 20.213, 'Ka2': 20.070, 'Kb1': 22.720, 'Kb2': 23.169, 'Kb3': 22.695},
    'Ag': {'Ka1': 22.159, 'Ka2': 21.987, 'Kb1': 24.938, 'Kb2': 25.452, 'Kb3': 24.907},
    'Zr': {'Ka1': 15.772, 'Ka2': 15.688, 'Kb1': 17.665, 'Kb2': 17.967, 'Kb3': 17.651},
    'Mo': {'Ka1': 17.476, 'Ka2': 17.371, 'Kb1': 19.605, 'Kb2': 19.962, 'Kb3': 19.587},
    'Cd': {'Ka1': 23.170, 'Ka2': 22.980, 'Kb1': 26.091, 'Kb2': 26.639, 'Kb3': 26.057},
    'Zn': {'Ka1': 8.637, 'Ka2': 8.614, 'Kb1': 9.570, 'Kb2': 9.656, 'Kb3': 9.570},
    'Fe': {'Ka1': 6.403, 'Ka2': 6.399, 'Kb1': 7.057, 'Kb2': np.nan, 'Kb3': 7.057},
}

# Function to compute weighted mean, ignoring NaN and None values.
def weighted_mean(values, weights = None):
```

```

    # Filter out None and NaN values.
    vals = np.array([v for v in values if v is not None and np.isfinite(v)],
dtype=float)

    # If no valid values, return NaN.
    if vals.size == 0:
        return np.nan

    # Use equal weights, when no weights are given.
    if weights is None:
        return float(np.mean(vals))

    # Remove weights corresponding to invalid values.
    ws = np.array([w for v, w in zip(values, weights) if v is not None and np.
isfinite(v)], dtype=float)
    if ws.size == 0 or np.sum(ws) == 0:
        return float(np.mean(vals))

    # Return weighted mean.
    return float(np.sum(vals * ws) / np.sum(ws))

# Weighted mean of Ka lines.
# Ka1:Ka2 intensity ratio is approximately 2:1.
def ka_mean(el, weights = [2.0, 1.0]):
    d = theory.get(el, {})
    return weighted_mean([d.get('Ka1'), d.get('Ka2')], weights = weights)

# Weighted mean of Kb lines.
def kb_mean(el, weights = None):
    d = theory.get(el, {})
    vals = [d.get('Kb1'), d.get('Kb2'), d.get('Kb3')]
    if weights is None:
        # equal weights, NaN are ignored
        return weighted_mean(vals)
    else:
        return weighted_mean(vals, weights = weights)

# Convert dict -> tidy DataFrame.
rec = []
for el, lines in theory.items():
    rec.append({'Element': el,
'E_th_Ka_mean': ka_mean(el), # Ka1:Ka2 is approximately 2:1.
'E_th_Kb_mean': kb_mean(el)})
theory_df = pd.DataFrame(rec)

# Merge theoretical values by Element.
df2 = df.merge(theory_df, on = 'Element', how = 'left')

```



```

# Normalize the Line labels.
def norm_line(s):
    s = str(s).lower().replace(" ", "").replace("_", "")
    if "ka+b" in s or ("ka" in s and "kb" in s):
        return "ka+b"
    if "ka" in s:
        return "ka"
    if "kb" in s:
        return "kb"
    return s # fallback

df2["Line_norm"] = df2["Line"].apply(norm_line)

# What to display as "theoretical energy" (string) depending on the line:
# - For K_a : show the weighted Ka mean
# - For K_b : show Kb1
# - For K_a+b: show "Ka_mean / Kb1" (so you see both); if you want a single
    ↪ centroid estimate,
#           you could compute one using an assumed K/K intensity ratio.
def choose_theory_display(row):
    if pd.isna(row["E_th_Ka_mean"]):
        return np.nan
    if row["Line_norm"] == "ka":
        return f"{row['E_th_Ka_mean']:.3f}"
    elif row["Line_norm"] == "kb":
        return f"{row['E_th_Kb_mean']:.3f}"
    elif row["Line_norm"] == "ka+b":
        return f"{row['E_th_Ka_mean']:.3f} / {row['E_th_Kb_mean']:.3f}"
    else:
        return np.nan

df2["E_th (keV)"] = df2.apply(choose_theory_display, axis = 1)

# If you want a *single* theoretical number also for K_a and K_b rows (numeric),
# we can keep numeric helpers and delta (measured - theoretical) where
    ↪ applicable:
def numeric_theory(row):
    E_a = row.get("E_th_Ka_mean", np.nan)
    E_b = row.get("E_th_Kb_mean", np.nan)

    if row["Line_norm"] == "ka":
        return E_a
    if row["Line_norm"] == "kb":
        return E_b
    if row["Line_norm"] == "ka+b":
        # assume intensity ratio I/I = r

```

```

    r = 0.15 # typical; adjust per element if needed
    if np.isfinite(E_a) and np.isfinite(E_b):
        return (E_a + r * E_b) / (1.0 + r)
    else:
        return np.nan
return np.nan

df2["E_th_num"] = df2.apply(numeric_theory, axis=1)
df2["ΔE (meas-th)"] = df2["CENTROID"] - df2["E_th_num"]

# Columns to display: the original columns + the theoretical values and delta.
# Improve the headers.
cols_out = ["ID", "Element", "Line", "CENTROID", "unc_CENTROID", "E_th (keV)",
    ↪ "ΔE (meas-th)"]
headers_out = ["Vzorek", "Prvek", "Čára", "E (keV)", "nej. E (keV)", "E_th
    ↪ (keV)", "ΔE (keV)"]

# Print the requested table.
print(df2.to_string(index = False, col_space = 8, columns = cols_out, header =
    ↪ headers_out))

```

Vzorek	Prvek	Čára	E (keV)	nej. E (keV)	E_th (keV)	ΔE (keV)
1	Cu	K _a +b	8.12	0.001314	8.039 / 8.928	-0.035246
2	Sn	K _a	25.24	0.001464	25.191	0.048667
2	Sn	K _b	28.55	0.003143	28.675	-0.124667
3	Rh	K _a	20.13	0.001777	20.165	-0.035333
3	Rh	K _b	22.72	0.003765	22.861	-0.141333
4	Pb	L _a	10.57	0.003087	NaN	NaN
4	Pb	L _b	12.62	0.003782	NaN	NaN
5	Ag	K _a	22.07	0.001341	22.102	-0.031667
5	Ag	K _b	24.94	0.002479	25.099	-0.159000
5	Cu	K _a +b	8.35	0.005242	8.039 / 8.928	0.194754
6	Zr	K _a	15.74	0.001275	15.744	-0.004000
6	Zr	K _b	17.66	0.002530	17.761	-0.101000
11	Cd	K _a	23.09	0.001710	23.107	-0.016667
11	Cd	K _b	26.11	0.003563	26.262	-0.152333
9	Mo	K _a	17.42	0.001760	17.441	-0.021000
9	Mo	K _b	19.63	0.003522	19.718	-0.088000

Úkol: Vyřadte slitiny z výše uvedeného DataFrame. Vykreslete intenzity jednoprvkových vzorků jako funkci protonového čísla Z prvku. Intenzita je součtem intenzit spektrálních čar K_α a K_β . Pokud nejsou přítomny, odeberte vzorek. K filtrování můžete použít buňku níže. Pokud chcete filtrování provést jiným způsobem (např. ručně), můžete vytvořit nový DataFrame libovolným způsobem. Pojmenujte jej `df_sum_intensities`, aby buňky pro vykreslení fungovaly.

```

[6]: import numpy as np

# Exclude alloys from the DataFrame.

```

```

# Alloys are samples that contain more than one chemical element.
# I.e. exclude samples that have the same sample number but the chemical
    ↪ elements are different in rows with the same sample number.
# Group by Sample and Element and count unique elements per sample
sample_element_counts = df.groupby('ID')['Element'].nunique()

# Get samples that have exactly one unique element
single_element_samples = sample_element_counts[sample_element_counts == 1].index

# Filter DataFrame to keep only those samples
df_single_element = df[df['ID'].isin(single_element_samples)]

# Filter DataFrame to keep only samples with K lines
df_single_element = df_single_element[df_single_element['Line'].str.
    ↪ contains('K')]

# For each sample, calculate the sum of the alpha and beta spectral line
    ↪ intensities.
# Create a new DataFrame with just one row per samples, and the sum of the
    ↪ intensities.
# Sum the uncertainties in quadrature.
df_sum_intensities = (df_single_element.groupby('ID', as_index=False)
    .agg({
        'Element': 'first',
        'Z': 'first',
        'Intensity': 'sum',
        'unc_Intensity': lambda x: np.sqrt(np.sum(x**2))
    }))

# TODO Finally, remove samples if there are doubts about the measurements:
df_sum_intensities = df_sum_intensities.loc[~df_sum_intensities['Element'].
    ↪ isin(['Gd', 'Pu'])]

# Print the result of the filtering and grouping.
print(df_sum_intensities.round(3).to_string(index=False, col_space=10))

```

ID	Element	Z	Intensity	unc_Intensity
1	Cu	29	33.651	0.396
2	Sn	50	125.237	0.890
3	Rh	45	90.547	0.865
6	Zr	40	78.382	0.659
9	Mo	42	108.693	0.998
11	Cd	48	132.016	1.097

Napišme si obecnější funkci pro vykreslení grafu.

```
[7]: import matplotlib as mpl
import matplotlib.pyplot as plt

def plot(x, y, y_err, x_fit, y_fit, y_fit_lower, y_fit_upper, xlabel, ylabel):

    # draw x, y with error bars
    plt.errorbar(x, y, y_err, fmt='o', label='Data', color='black')

    # draw the fit function and its uncertainty band
    plt.fill_between(x_fit, y_fit_lower, y_fit_upper, color='red', alpha=0.3)

    # create a legend entry for the fit function and its uncertainty band
    line_with_band = mpl.lines.Line2D([], [], color='red', label='Fit',
    ↪ linestyle='-', linewidth=2)
    band = mpl.patches.Patch(color='red', alpha=0.3, label='Fit uncertainty')

    # get the current legend handles and labels
    handles, labels = plt.gca().get_legend_handles_labels()
    plt.legend(handles=handles + [(line_with_band, band)], labels=labels +
    ↪ ['Fit'])

    # finally, plot
    plt.plot(x_fit, y_fit, 'r-')
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.show()

    return
```

A fitujme naše data.

```
[8]: # Fit the intensity on Z dependence with a function.
# TODO define the return value of the function.
# Don't forget to use the numpy mathematical functions which work with arrays.
def intensity_law(Z, a, b):
    return Z**a*b

# Determine the parameters and their uncertainties.
# Plot the data points and the fitted function in the same plot.
# Don't forget to take the correlation between the parameters into account when
    ↪ calculating the fit function uncertainty!

from scipy.optimize import curve_fit
import uncertainties
import uncertainties.unumpy
```

```

# Fit the power law function to the data
nom, cov = curve_fit(intensity_low, df_sum_intensities['Z'],
    ↪df_sum_intensities['Intensity'], sigma=df_sum_intensities['unc_Intensity'],
    ↪absolute_sigma=True)

# TODO Calculate the uncertainties of the fit parameters. Print the fit result.
a, b = uncertainties.correlated_values(nom, cov)
print(f'a = {a:.2uP}')
print(f'b = {b:.2uP}')
print(f'correlation coefficient: {cov[0, 1] / np.sqrt(cov[0, 0] * cov[1, 1]):.
    ↪3f}')

# Evaluate the fit function on 100 values of Z-grid
Z_fit = np.linspace(df_sum_intensities['Z'].min(), df_sum_intensities['Z'].
    ↪max(), 100)
I_fit = intensity_low(Z_fit, a, b)
I_fit_nom = uncertainties.unumpy.nominal_values(I_fit)
I_fit_std = uncertainties.unumpy.std_devs(I_fit)

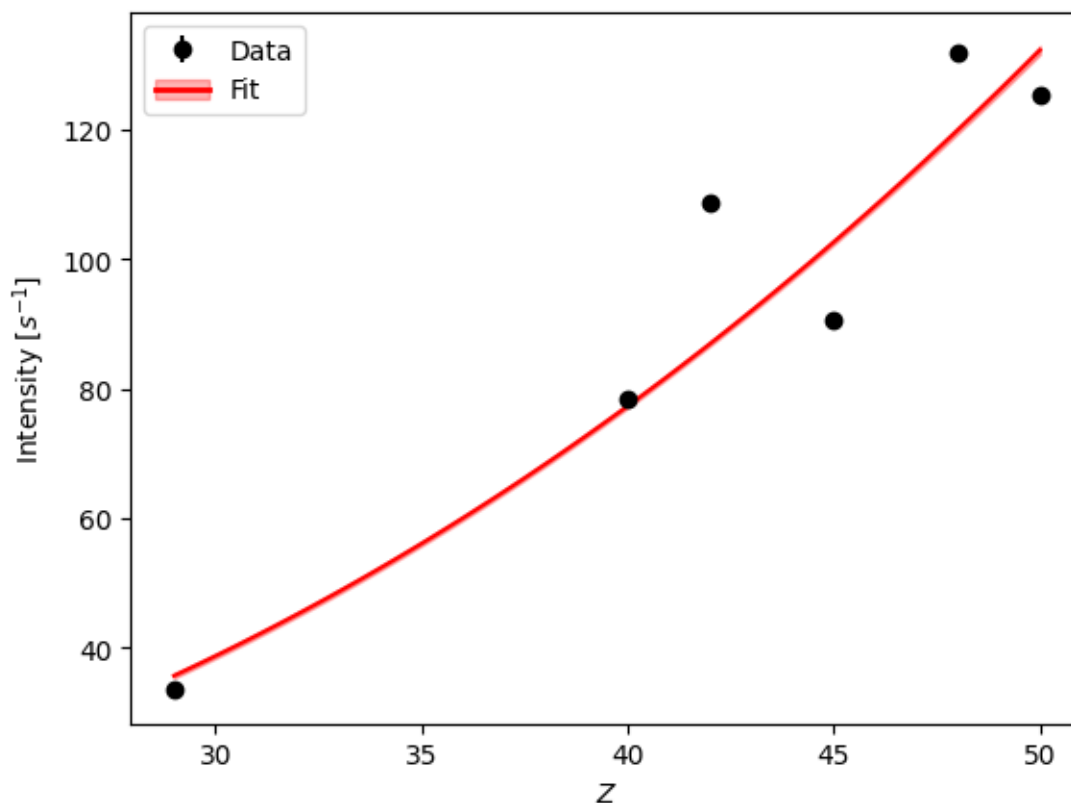
# Plot the data and the fit function
plot(df_sum_intensities['Z'], df_sum_intensities['Intensity'],
    ↪df_sum_intensities['unc_Intensity'],
    Z_fit, I_fit_nom, I_fit_nom - I_fit_std, I_fit_nom + I_fit_std, r'$Z$',
    ↪r'$\mathrm{Intensity} \ [s^{-1}]$')

```

a = 2.408±0.022

b = 0.01075±0.00091

correlation coefficient: -0.999



Úkol: Pro slitiny určete kvantitativně složení vzorků.
Vytvořme si separátní DataFrame a vypišme si ho.

```
[10]: # Filter DataFrame to keep only alloys.
df_alloys = df[~df['ID'].isin(single_element_samples)]

# Print the DataFrame with alloys.
print(df_alloys.round(3).to_string(index=False, col_space=7, columns=cols,
    ↪header=headers))
```

Vzorek	Prvek	Z	E (keV)	nej. E (keV)	Čára	I (s ⁻¹)	nej. I (s ⁻¹)
5	Ag	47	22.07	0.001	K_a	47.717	0.360
5	Ag	47	24.94	0.002	K_b	15.141	0.301
5	Cu	29	8.35	0.005	K_a+b	7.620	0.302

Vzhledem k malému množství dat volíme pro slitiny manuální zpracování:

```
[11]: # Don't do anything fancy with the alloys, just copy their values.

# TODO rewrite everything below according to Your findings

#### EXAMPLE: ####
```

```

# imagine we have a sample #420 made of Og and Tc
# hardcode their intensities, already in units per second
I_Og = uncertainties.ufloat(7.620, 0.302)
I_Tc = uncertainties.ufloat(47.717, 0.360) + uncertainties.ufloat(15.141, 0.301)

# Print the intensities of Og and Tc in the alloy
print(f'Intenzita Og ve slitině: ({I_Og:.2uP}) s-1')
print(f'Intenzita Tc ve slitině: ({I_Tc:.2uP}) s-1')

# Intensities of pure Og and Tc samples.
# Og shows case when we've measured pure sample,
# Tc the case when we have not, so we need to estimate it from the fitted
↪ dependence
I_Og_ref = uncertainties.ufloat(33.651, 0.396)
I_Tc_ref = intensity_law(47, a, b)

# Print the intensities of pure samples of Og a Tc.
print(f'Intenzita čistého vzorku Og: ({I_Og_ref:.2uP}) s-1')
print(f'Intenzita čistého vzorku Tc: ({I_Tc_ref:.2uP}) s-1')

# TODO calculate the normalisation factor if need be
norm = 100/((I_Og / (I_Og_ref))+(I_Tc / (I_Tc_ref)))

# Print the fraction of Og and Tc in the alloy
print(f'Frakce Og ve slitině: {norm * I_Og / (I_Og_ref):.2uP}')
print(f'Frakce Tc ve slitině: {norm * I_Tc / (I_Tc_ref):.2uP}')

```

```

Intenzita Og ve slitině: (7.62±0.30) s-1
Intenzita Tc ve slitině: (62.86±0.47) s-1
Intenzita čistého vzorku Og: (33.65±0.40) s-1
Intenzita čistého vzorku Tc: (113.99±0.45) s-1
Frakce Og ve slitině: 29.11±0.87
Frakce Tc ve slitině: 70.89±0.87

```

Úkol: Diskutujte měření a výsledky. Jakou funkci jste zvolili pro fitování dat a proč? Jaké vlivy jste při Vašem fitu zanedbali?

Řešení: # Diskuse

Pro vykreslení intenzity jednoprvkových vzorků jako funkci protonového čísla Z prvku jsme použili funkci s předpisem $y = Z^a \cdot b$, kde n se pohybuje mezi 4 a 5. V našem případě nám vyšlo přibližně 2.5. Důvodem mohou být různé tloušťky použitých vzorků (při detailním zvětšení) a s tím spojená různá pronikavost rentgenového záření. Dalším faktorem ovlivňující měření je různá účinnost detektoru pro různé energie. Nakonec také tzv. fluorescenční yield, který udává poměr vyzářených fotonů k počtu absorbovaných. Tedy ne všechny excitované atomy uvolní energii v podobně fluorescenčního fotonu, ale může dojít k uvolnění energie jinou formou. To může záviset například i na atomovém čísle.

Korelační koeficient fitu je velmi blízko -1. To znamená že zde dochází k silné negativní závislosti.

Hodnoty tedy dobře odpovídají modelu.

Jak jsme diskutovali výše, systematická chyba dominovala nad statistickou. Přesnější kalibrací by mohlo být teoreticky možné snížit systematickou chybu.

Rozlišení použitého detektoru neumožňovalo v určitých případech rozeznat mezi jednotlivými singlety a místo toho jsme pozorovali multiplet.

Úkol: Sepiš Závěr.

Řešení: # Závěr

V rámci úlohy jsme provedli energetickou kalibraci gama-spektrometru pomocí alfa-zářiče ^{241}Am a následně měřili charakteristické rentgenové spektrum několika vzorků. Na základě energií charakteristických čar (K a L série) byly jednoznačně identifikovány prvky Cu, Sn, Rh, Pb, Ag, Zr, Cd a Mo.

Pro jednoprvkové vzorky jsme určili závislost intenzity záření na protonovém čísle Z . Fitování dat funkcí typu $y(Z) = a \cdot Z^n$ poskytlo exponent $a = 0.01075 \pm 0.00091$ a koeficient $b = 2.408 \pm 0.022$ s korelačním koeficientem -0.999, což ukazuje velmi dobrou shodu modelu s daty.

U slitiny Cu–Ag jsme kvantitativně určili hmotnostní zastoupení obou fází, přičemž výsledek odpovídal očekávaným hodnotám s převažujícím podílem Ag (~71 %). Statistické chyby se ukázaly jako zanedbatelné oproti systematickým, což potvrzuje dostatečnou délku měření a stabilitu přístroje.

6 Literatura

[1] Thompson, A. C., *et al.*, X-Ray Data Booklet, 3rd ed. (rev. 2009). Lawrence Berkeley National Laboratory, Berkeley, CA. Dostupné na https://xdb.lbl.gov/Section1/Table_1-2.pdf .

[2] Basunia, M. S., Nucl. Data Sheets 107, 3323 (2006). DOI: 10.1016/j.nds.2006.07.001 , dostupné na <https://www.nndc.bnl.gov/ensdf/> .

[1] MFF. Studijní text: [Online]. [cit. 19. října 2025], dostupné na https://physics.mff.cuni.cz/vyuka/zfp/_media/zadani/texty/txt_403.pdf3